
Lesson 1 Lab Sheet - Building and Describing Sequences

What you will do today

In the theoretical part of Lesson 1, we discussed why life-course researchers may want to study complete trajectories (instead of isolated events). Here is the practical session related to that.

By the end of this session, you should be able to:

- Identify the unit, time axis, time unit and states in a longitudinal dataset
- Reshape person-period data from long to wide format
- Create a state-sequence object with `TraMineR`
- Produce and interpret a sequence index plot and a state distribution plot
- Calculate time spent in states, number of transitions, but also, if we have enough time today, entropy and sequence complexity
- Start your mini-project and produce the first descriptive figure :)

We use the **employment trajectories from ages 18 to 30** as an example. During the final part of the session, you will apply the same commands to the research question you choose for your mini-project.

Please note that the emphasis will not be put on knowing the commands, or learning how to use R. Feel free to copy-paste the commands, but understand what they really mean and what they do so you don't misuse them.

0. PACKAGES

Objective: Run the package-loading commands.

Run this only if the packages are not already installed:

```
packages <- c(
  "TraMineR",
  "WeightedCluster",
  "dplyr",
  "tidyr",
  "ggplot2"
)

new_packages <- packages[
  !sapply(packages, requireNamespace, quietly = TRUE)
]

if (length(new_packages) > 0) {
  install.packages(new_packages)
}
```

Then load the packages needed today:

```
library(TraMineR)
library(dplyr)
library(tidyr)
library(ggplot2)
```

`library(package_name)` attaches an installed package to the current R session.

- `TraMineR` : sequence construction, description and later distance analysis.
- `dplyr` : data manipulation.
- `tidyr` : reshaping between long and wide formats.
- `ggplot2` : ordinary statistical graphics when `TraMineR`-specific plots are not needed.

`install.packages()` normally needs to be run only once on a computer. `library()` needs to be run again in each new R session.

1. FIND THE DATA

Objective: Check the working directory and whether R can find the data files.

```
getwd()
```

`getwd()` means “get working directory”. Relative paths such as `data/sequence_course_long.csv` are interpreted relative to this folder. Ideally, R should currently be working inside the `sequence_course_dataset` folder. use `setwd()` if the working directory needs to be changed.

```
stopifnot(file.exists("data/sequence_course_long.csv"))
stopifnot(file.exists("data/sequence_course_person.csv"))
stopifnot(file.exists("data/sequence_course_wide.csv"))
```

`file.exists()` asks whether a file can be found. `stopifnot()` stops the script if its condition is `FALSE`. All three files should be found.

2. IMPORT THE THREE TEACHING FILES

Objective: Import `long` , `person` , and `wide_backup` .

```
long <- read.csv(
  "data/sequence_course_long.csv",
  stringsAsFactors = FALSE
)

person <- read.csv(
  "data/sequence_course_person.csv",
  stringsAsFactors = FALSE
)

wide_backup <- read.csv(
  "data/sequence_course_wide.csv",
  stringsAsFactors = FALSE
)
```

Note: long is person-age data; person is person-level data.

read.csv() reads a CSV file and stores it as a data frame.

OBJECTS:

- long = one row per person-age.
- person = one row per person.
- wide_backup = one row per person with sequences already in wide format.

We begin from LONG data because converting longitudinal observations into sequences is an important part of the analytical process.

The wide file is just a safety net. You might not need to use it.

3. FIRST LOOK AT THE DATA

Objective: Inspect dimensions, first rows, and variable types.

```
dim(long)
dim(person)

head(long)
head(person)

str(long)
str(person)
```

dim() returns number of rows and columns.

head() prints the first six rows.

Always inspect data before analysing them. Never assume the import worked exactly as expected.

str() shows the structure of an object: variable names, data types and a small preview.

Check that id and age should be numeric/integer. employment_state, family_state and housing_state should be character.

Questions:

Write short answers.

1. How many individuals are in the dataset?
2. How many person-age observations are in the long file?
3. What age range is observed?
4. How many observations does each person contribute?
5. What does one row represent in long and in person ?

4. CHECK THE LONGITUDINAL STRUCTURE

Objective: Check the number of people, ages, observations per person, and missing states.

```
n_distinct(long$id)
range(long$age)
```

n_distinct() counts unique values. range() gives minimum and maximum.

Questions: How many people do we have? What is the observation window?

```
obs_per_person <- long %>%
  count(id, name = "n_observations")
```

%>% is the pipe. It passes the object on the left into the next command.

count(id) creates one row per id and counts how many rows that id has.

OBJECT: obs_per_person = one row per person + number of observations.

```
table(obs_per_person$n_observations)
```

Question: Do all individuals contribute the same number of ages? In other term, does everyone observed from age 18 to 30?

Note: Equal-length sequences are convenient here, but sequence data in real research may contain censoring, gaps or missing states. Those cases require explicit methodological decisions.

```
sum(is.na(long$employment_state))
sum(is.na(long$family_state))
sum(is.na(long$housing_state))
```

is.na() marks missing values TRUE/FALSE. sum(TRUE/FALSE) counts the TRUE values.

Question: Why should we inspect missingness BEFORE constructing the sequence object?

5. INSPECT THE EMPLOYMENT ALPHABET

Objective: List and count the employment states.

Question: What are the five employment states?

```
table(long$employment_state)
```

`table()` counts observations in each category.

NOTE: In sequence analysis, the set of possible states is called the ALPHABET.

Our employment alphabet is: education, full_time, part_time, unemployed, inactive_care

```
sort(unique(long$employment_state))
```

`unique()` returns the distinct observed values. `sort()` puts them in order.

It is good practice to verify that the coding in the data corresponds to the conceptual alphabet we intend to analyse.

Note: The alphabet is the set of possible sequence states.

6. MOVE FROM LONG TO WIDE FORMAT

Objective: Reshape employment data from long to wide.

```
employment_wide <- long %>%  
  select(id, age, employment_state) %>%  
  mutate(age = paste0("emp_", age)) %>%  
  pivot_wider(  
    names_from = age,  
    values_from = employment_state  
  ) %>%  
  arrange(id)
```

READ THIS PIPE FROM TOP TO BOTTOM:

`select(...)` : keeps only the variables we need.

`mutate(age = paste0("emp_", age))` : changes 18, 19, 20 ... into "emp_18", "emp_19", "emp_20" ... These values will become the names of the sequence columns.

`pivot_wider(...)` : converts one row per person-age into one row per person. `names_from = age` says: values of age become new COLUMN NAMES. `values_from = employment_state` says: fill those columns with the states.

`arrange(id)` : sorts rows by id.

OBJECT: `employment_wide` now contains one person's complete employment history on one row.

```
head(employment_wide)  
dim(employment_wide)
```

Question: What does `emp_22` mean? What is the employment state for [person 1, `emp_22`]?

Note: The information stays the same, you only change storage format.

7. VERIFY THE RESHAPE

Objective: Compare your reshaped data with the supplied backup.

```
all(employment_wide$id == wide_backup$id)
```

`all()` is TRUE only if every comparison is TRUE.

```
all(employment_wide$emp_18 == wide_backup$emp_18)
all(employment_wide$emp_30 == wide_backup$emp_30)
```

8. DEFINE THE STATE SEQUENCE OBJECT

Objective: Create `emp_seq` with `seqdef()`.

```
emp_cols <- paste0("emp_", 18:30)
```

`paste0()` creates a character vector: "emp_18", "emp_19", ... "emp_30".

We can select 13 sequence columns without typing every name.

```

employment_alphabet <- c(
  "education",
  "full_time",
  "part_time",
  "unemployed",
  "inactive_care"
)

employment_codes <- c(
  "EDU",
  "FT",
  "PT",
  "UN",
  "IC"
)

employment_labels <- c(
  "Education",
  "Full-time employment",
  "Part-time employment",
  "Unemployment",
  "Inactive/care"
)

emp_seq <- seqdef(
  data = employment_wide[, emp_cols],
  alphabet = employment_alphabet,
  states = employment_codes,
  labels = employment_labels,
  id = employment_wide$id,
  cnames = 18:30,
  xtstep = 2,
  tick.last = TRUE
)

```

`seqdef()` converts an ordinary table of successive states into a TraMineR state-sequence object (class “`stslist`”).

ARGUMENTS: `data` = the 13 columns containing the states.

`alphabet` = the raw possible values appearing in the data.

`states` = short display codes. These are labels for the alphabet, not new states.

`labels` = longer human-readable labels used in legends.

`id` = identifiers attached to the sequence rows.

`cnames` = labels for the sequence positions. Here they are ages 18...30.

`xtstep` = 2 ask plots to show every second tick where possible.

`tick.last` = TRUE ensure the final position is represented on the axis.

OBJECT: `emp_seq` is not just a data frame. It stores both the sequence values and metadata such as the alphabet, labels and plotting information.

```
class(emp_seq)
alphabet(emp_seq)
summary(emp_seq)
```

Note: We constructed an age-aligned annual employment sequence from age 18 to age 30, using five mutually exclusive states. `emp_seq` is a TraMineR sequence object.

Question: What does one sequence position represent?

9. LOOK AT INDIVIDUAL SEQUENCES

Objective: Print a few individual sequences.

```
emp_seq[1:8, ]
```

`[rows, columns]` subsets an object. Here we request the first 8 complete sequences.

Reading a few sequences as text before plotting hundreds of them is often very useful. It keeps the analysis connected to actual life histories.

Question: Can you describe one trajectory in ordinary language?

10. SEQUENCE INDEX PLOT

Objective: Produce the sequence index plot.

```
n_transitions <- seqtransn(emp_seq)

seqIplot(
  emp_seq,
  sortv = n_transitions,
  main = "Employment trajectories, ages 18-30"
)
```

`seqIplot()` displays one horizontal trajectory per individual.

``sortv = n_transitions`` sorts individuals according to their number of state changes. This is not required; it simply makes one dimension of heterogeneity easier to see.

Check that:

- Are some histories dominated by one state?
- Where do unemployment and care interruptions occur?
- Are highly fragmented sequences concentrated together because of sorting?

An index plot retains individual histories. It can show heterogeneity, but with many individuals it can also become dense.

Question: What can you see here that a table of percentages would hide?

Note: One horizontal line is one person's trajectory.

11. STATE DISTRIBUTION PLOT

Objective: Produce the state distribution plot.

```
seqdplot(  
  emp_seq,  
  main = "Employment-state distribution by age"  
)
```

`seqdplot()` computes, at each age, the proportion of the sample in each state.

Check that:

- How quickly does education decline?
- When does full-time employment become dominant?
- At which ages are unemployment/care most visible?

Note: This plot is a sequence of cross-sectional distributions. At each age, it shows the proportion in each state. It shows aggregate composition, but it does NOT tell us whether the same people remain in a state or whether many individuals move in and out of it.

Question: Index plot versus distribution plot: what information does each retain, and what information does each lose?

12. COMPARE GROUPS DESCRIPTIVELY

Objective: Compare state distributions by parental education.

```
person_for_seq <- person %>%  
  arrange(id)  
  
stopifnot(all(person_for_seq$id == employment_wide$id))  
  
seqdplot(  
  emp_seq,  
  group = person_for_seq$parental_education,  
  main = "Employment trajectories by parental education"  
)
```

`group=` tells TraMineR to calculate a separate state-distribution plot for each level of the grouping variable.

This is our first example of using a sequence outcome to compare social groups.

IMPORTANT: This is descriptive. We have not estimated a causal effect.

Question: Do groups differ mainly in

- duration?

- timing?
- state composition?

13. TIME SPENT IN EACH STATE

Objective: Calculate time spent in each state.

```
state_duration <- seqistatd(emp_seq)
head(state_duration)
```

`seqistatd()` returns, for each individual, the amount of sequence time spent in each state.

Because our time unit is one year, these values can be read approximately as years in each state over ages 18-30.

OBJECT: `state_duration` = matrix with one row per person and one column per state.

```
colMeans(state_duration)
```

`colMeans()` calculates the average of each column. It gives a simple description of average cumulative exposure.

Duration is cumulative exposure to a state. Duration is important, but it loses order. Two people can both experience 3 years of unemployment: one in a single spell, another in several interruptions.

14. NUMBER OF TRANSITIONS

Objective: Calculate the number of transitions.

```
n_transitions <- seqtransn(emp_seq)
summary(n_transitions)
```

`seqtransn()` counts changes from one state to another.

Example: E E E U U E

changes E->U and U->E = 2 transitions.

It is one simple indicator of instability or fragmentation.

CAUTION: A transition is any state change. A transition is not automatically “bad”. Education -> employment is also a transition.

Question: Why should we avoid interpreting “more transitions” as “more disadvantage”?

15. ENTROPY AND COMPLEXITY

Objective: Calculate entropy and complexity.

Entropy measures how diverse the states are within a sequence. It is higher when a person spends time in several different states, especially when that time is distributed relatively evenly across them. A sequence spent entirely in one state has very low entropy.

Complexity measures how complex the whole sequence is, taking into account both state diversity and the number of transitions between states. So two people can have similar entropy, but the one who switches back and forth more often will generally have greater sequence complexity

```
entropy <- seqient(emp_seq)
complexity <- seqici(emp_seq)

summary(entropy)
summary(complexity)

cor(entropy, complexity)
```

These reduce the trajectory to numerical summaries.

- `seqient()` measures within-sequence entropy: how diverse the states experienced by an individual are.
- `seqici()` calculates a sequence complexity index that combines information on state diversity and transitions.
- `cor()` calculates the linear correlation between two numerical variables.

Note: Entropy and complexity are related but not identical ideas. A sequence can contain several states without switching constantly, or switch repeatedly among a small set of states.

Questions: - Why might a single index be useful?

- What information is lost when a whole trajectory becomes one number?

16. BUILD A SMALL ANALYTICAL DATA FRAME

Objective: Combine background variables with sequence indicators.

```
lesson1_summary <- person_for_seq %>%
  mutate(
    n_transitions = n_transitions,
    entropy = entropy,
    complexity = complexity,
    unemployment_years = rowSums(
      employment_wide[, emp_cols] == "unemployed"
    )
  )
```

`mutate()` adds variables.

`rowSums(condition)` works because: `employment_wide[, emp_cols] == "unemployed"` creates a TRUE/FALSE matrix. TRUE is treated as 1 and FALSE as 0 when summed.

OBJECT: `lesson1_summary` combines background information and simple sequence indicators for each person.

```
head(lesson1_summary)

lesson1_summary %>%
  group_by(parental_education) %>%
  summarise(
    mean_unemployment = mean(unemployment_years),
    mean_transitions = mean(n_transitions),
    mean_complexity = mean(complexity),
    .groups = "drop"
  )
```

`group_by()` temporarily defines groups. `summarise()` collapses each group into summary statistics.

This is an example of translating complex sequences into numerical descriptors that can later be used in substantive comparisons.

17. SAVE OBJECTS FOR LATER LESSONS

Objective: Save today's main objects.

```
dir.create("outputs", showWarnings = FALSE)

saveRDS(emp_seq, "outputs/emp_seq.rds")
saveRDS(employment_wide, "outputs/employment_wide.rds")
saveRDS(lesson1_summary, "outputs/lesson1_summary.rds")
```

`dir.create()` creates a folder. `showWarnings=FALSE` avoids a warning if the folder already exists.

`saveRDS()` saves ONE R object to disk.

In later lessons we can retrieve the object with `readRDS()` instead of reconstructing it every time.

Reproducible research still requires keeping the code that created these objects. Saving an object is a convenience, not a substitute for a script.

18. QUESTIONS/ANSWERS TO REMEMBER

- What is a sequence in this dataset?
- What is the difference between an index plot and a state distribution plot?
- What is the difference between duration and number of transitions?
- Why do the time axis and time unit matter?

MINI-PROJECT — TODAY'S TASK

Report section(s): 1. Research question and rationale; 2. Data and sequence construction

1. Choose your research question and sequence domain.
2. Define your population, time axis, time unit, and states.
3. Create your own sequence object.
4. Produce one descriptive sequence figure.
5. Write 2-3 sentences describing what you see.

Save before leaving or finish before next lesson: your mini-project sections 1 and 2 and first figure.